

AUVEN FASHION API DOCUMENTATION

MERN Full Stack E-commerce Platform

Table of Contents

1. Project Overview
 2. Architecture Overview
 3. Database Overview
 4. Environment Configuration
 5. Authentication
 6. API Overview Table
 7. Detailed API Documentation
 - Authentication APIs
 - User APIs (Customer)
 - Product APIs
 - Category APIs
 - Cart APIs
 - Customizer APIs
 - Order APIs
 - Payment APIs
 - Address APIs
 - Coupon APIs
 - Homepage Content APIs
 - Admin APIs
 - Other APIs
 8. Error Handling
 9. Postman Testing Guide
 10. API Workflow
 11. Implementation Status
-

1. Project Overview

AUVEN Fashion is a customisable T-Shirt clothing brand running on a MERN full-stack architecture. This document provides an exhaustive reference for all the backend REST APIs actually implemented in the system.

2. Architecture Overview

- **Backend Framework:** Node.js with Express.js
 - **Database:** MongoDB (using Mongoose ODM)
 - **Authentication:** JSON Web Tokens (JWT)
 - **File Uploads:** Multer & Cloudinary
 - **Payments:** Stripe API & Cash On Delivery
 - **Routing:** Express Router
-

3. Backend Data Model Overview

The backend uses Mongoose to interact with MongoDB.

- **userModel:** Stores name, email, password, cartData (JSON object), and an array of addresses.
 - **productModel:** Stores product name, description, price, image (array), category, subcategory, sizes, inventory array (size, colour, quantity), bestseller, and date.
 - **orderModel:** Stores userId, items, amount, address, status, paymentMethod, payment boolean, couponCode, discountAmount, and date.
 - **designModel:** Stores predefined customizer designs (name, image, category, active).
 - **couponModel:** Stores discount codes (code, discountPercentage, minimumOrderAmount, maximumDiscountAmount, expiryDate, active).
 - **homepageContentModel:** Stores dynamic UI configurations (type, title, image, buttonText, productIds, etc.).
-

4. Environment Configuration

Variables required for the backend to run:

- PORT
- MONGODB_URI
- JWT_SECRET
- CLOUDINARY_NAME
- CLOUDINARY_API_KEY
- CLOUDINARY_SECRET_KEY
- ADMIN_EMAIL
- ADMIN_PASSWORD
- STRIPE_SECRET_KEY
- CORS_ORIGIN

(Note: Actual secrets and passwords are kept entirely out of this documentation for security.)

5. Authentication

The application implements two distinct authorization levels: User and Admin.

- **User Authentication:** Requires a valid JWT token signed with `JWT_SECRET`. Token payload contains `{ id: user._id }`.
- **Admin Authentication:** Requires a token generated by signing `ADMIN_EMAIL + ADMIN_PASSWORD` with `JWT_SECRET`.

Middleware Used:

- `authUser`: Validates JWT token from `token` header. Decodes it and appends `userId` to the request body.
- `adminAuth`: Validates JWT token from `token` header against the hardcoded Admin credentials.

6. API Overview Table

Method	Endpoint	Authentication	Role	Description
POST	/api/user/register	Not Required	Public	Register a new user
POST	/api/user/login	Not Required	Public	User login
POST	/api/user/admin	Not Required	Public	Admin login
GET	/api/user/profile	Required	User	Get user profile
PUT	/api/user/profile	Required	User	Update user profile
PUT	/api/user/change-password	Required	User	Change user password
POST	/api/user/addresses	Required	User	Add new address
PUT	/api/user/addresses/:addressId	Required	User	Update address
DELETE	/api/user/addresses/:addressId	Required	User	Delete address
PUT	/api/user/addresses/:addressId/default	Required	User	Set default address
POST	/api/product/add	Required	Admin	Add new product
POST	/api/product/remove	Required	Admin	Remove product
POST	/api/product/single	Not Required	Public	Get single product info
GET	/api/product/list	Not Required	Public	Get all products
POST	/api/product/update	Required	Admin	Update existing product
POST	/api/cart/get	Required	User	Get user cart data
POST	/api/cart/add	Required	User	Add item to cart
POST	/api/cart/update	Required	User	Update item in cart
POST	/api/cart/addStandaloneCustomized	Required	User	Add customized t-shirt to cart
POST	/api/cart/updateStandaloneCustomized	Required	User	Update customized t-shirt in cart
POST	/api/order/list	Required	Admin	Get all orders
POST	/api/order/status	Required	Admin	Update order status
POST	/api/order/place	Required	User	Place COD order
POST	/api/order/stripe	Required	User	Place Stripe order
POST	/api/order/verifyStripe	Required	User	Verify Stripe payment
POST	/api/order/userorders	Required	User	Get user's orders
POST	/api/customizer/upload	Required	User	Upload customer design image
GET	/api/customizer/designs	Not Required	Public	Get active predefined designs
GET	/api/customizer/admin/designs	Required	Admin	Get all predefined designs
POST	/api/customizer/admin/designs	Required	Admin	Add predefined design
PUT	/api/customizer/admin/designs/:id	Required	Admin	Update predefined design
DELETE	/api/customizer/admin/designs/:id	Required	Admin	Delete predefined design
GET	/api/config/constants	Not Required	Public	Get pricing constants
GET	/api/admin/dashboard	Required	Admin	Get dashboard statistics
GET	/api/admin/customers	Required	Admin	Get all customers
GET	/api/admin/customers/:id	Required	Admin	Get single customer details
GET	/api/admin/customers/:id/orders	Required	Admin	Get single customer orders
POST	/api/coupon/validate	Required	User	Validate coupon code
GET	/api/admin/coupons	Required	Admin	Get all coupons
POST	/api/admin/coupons	Required	Admin	Create a new coupon
PUT	/api/admin/coupons/:id	Required	Admin	Update coupon
PUT	/api/admin/coupons/:id/toggle	Required	Admin	Toggle coupon active status
DELETE	/api/admin/coupons/:id	Required	Admin	Delete a coupon
GET	/api/homepage	Not Required	Public	Get active homepage content
GET	/api/admin/homepage	Required	Admin	Get all homepage content
POST	/api/admin/homepage	Required	Admin	Create homepage content
PUT	/api/admin/homepage/:id	Required	Admin	Update homepage content
PUT	/api/admin/homepage/:id/toggle	Required	Admin	Toggle content active status
DELETE	/api/admin/homepage/:id	Required	Admin	Delete homepage content

7. Detailed API Documentation

Authentication APIs

Register User

Method: POST
Endpoint: /api/user/register
Description: Registers a new user. Hashes the password and returns a JWT token.
Authentication: Not Required
Authorization: Public
Headers: None

Path Parameters: None
Query Parameters: None
Request Body:

```
{
  "name": "John Doe",
  "email": "johndoe@example.com",
  "password": "strongpassword123"
}
```

Success Response:

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1..."
}
```

HTTP Status: 200 OK

User Login

Method: POST
Endpoint: /api/user/login
Description: Login an existing user and returns a JWT token.
Authentication: Not Required
Authorization: Public
Request Body:

```
{
  "email": "johndoe@example.com",
  "password": "strongpassword123"
}
```

Success Response:

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1..."
}
```

Admin Login

Method: POST
Endpoint: /api/user/admin
Description: Login an admin by comparing credentials with env variables. Returns an admin JWT token.
Authentication: Not Required
Authorization: Public
Request Body:

```
{
  "email": "admin@example.com",
  "password": "adminpassword"
}
```

Success Response:

```
{
  "success": true,
  "token": "admin_jwt_token_here"
}
```

User APIs (Customer)

Get Profile

Method: GET
Endpoint: /api/user/profile
Description: Retrieves the logged-in user's profile information (excludes password).
Authentication: Required
Authorization: User
Headers:

Header Required	Description
token	Yes User JWT Token

Success Response:

```
{
  "success": true,
  "user": {
    "_id": "648a1b2c3d...",
    "name": "John Doe",
    "email": "johndoe@example.com",
    "addresses": [],
    "cartData": {}
  }
}
```

Add Address

Method: POST

Endpoint: /api/user/addresses

Description: Adds a new shipping address to the user's profile. Automatically sets it to default if it is the first address or isDefault is true.

Authentication: Required

Authorization: User

Request Body:

```
{
  "name": "Home",
  "phone": "9876543210",
  "addressLine1": "123 Main Street",
  "addressLine2": "Apt 4B",
  "city": "Chennai",
  "state": "Tamil Nadu",
  "postalCode": "600001",
  "country": "India",
  "isDefault": true
}
```

Success Response:

```
{
  "success": true,
  "message": "Address added successfully",
  "addresses": [ { ... } ]
}
```

(Note: Endpoints for Update Address, Delete Address, Set Default Address, Update Profile, and Change Password follow the identical structure utilizing the User ID provided via the authUser middleware).

Product APIs

Get All Products

Method: GET

Endpoint: /api/product/list

Description: Retrieves a list of all products in the database.

Authentication: Not Required

Authorization: Public

Success Response:

```
{
  "success": true,
  "products": [
    {
      "_id": "60d21b46...",
      "name": "Classic T-Shirt",
      "description": "Comfortable cotton t-shirt",
      "price": 499,
      "category": "Men",
      "subcategory": "Topwear",
      "sizes": ["S", "M", "L", "XL"],
      "inventory": [
        { "size": "M", "colour": "", "quantity": 10 }
      ],
      "image": ["https://cloudinary.url/image1.jpg"],
      "bestseller": true,
      "date": 1690000000000
    }
  ]
}
```

Get Single Product

Method: POST

Endpoint: /api/product/single

Description: Retrieves data for a specific product.

Authentication: Not Required

Authorization: Public

Request Body:

```
{
  "productId": "60d21b46..."
}
```

Cart APIs

Add To Cart

Method: POST

Endpoint: /api/cart/add

Description: Adds a specific quantity of a standard product (variant) to the user's cart. Performs rigorous inventory validation to ensure requested quantity doesn't exceed available stock.

Authentication: Required

Authorization: User

Request Body:

```
{
  "itemId": "60d21b46...",
  "size": "M",
  "quantity": 2
}
```

Success Response:

```
{
  "success": true,
  "message": "Added To Cart"
}
```

Error Response (Out of Stock):

```
{
  "success": false,
  "message": "Only 5 items are available for size M."
}
```

Customizer APIs

Upload Customer Design

Method: POST

Endpoint: /api/customizer/upload

Description: Uploads a user-provided image (PNG, JPG, WEBP) to Cloudinary for custom T-Shirt printing. Maximum size: 5MB.

Authentication: Required

Authorization: User

Headers:

Header	Required	Description
token	Yes	User JWT Token
Content-Type	Yes	multipart/form-data

Form Data:

- image: File (Binary)

Success Response:

```
{
  "success": true,
  "imageUrl": "https://res.cloudinary.com/.../uploads/custom1.png"
}
```

Add Standalone Customized T-Shirt to Cart

Method: POST

Endpoint: /api/cart/addStandaloneCustomized

Description: Adds a fully customized T-Shirt to the cart. Calculates dynamic pricing based on a base price + per-object pricing (Text/Design elements).

Authentication: Required

Authorization: User

Request Body:

```
{
  "customKey": "custom_1694301234",
  "colour": "#FFFFFF",
  "quantity": 1,
  "customization": {
    "objects": [
      { "type": "text", "text": "Hello World" },
      { "type": "image", "src": "https://cloudinary.com/..." }
    ]
  }
}
```

Success Response:

```
{
  "success": true,
  "message": "Customized T-Shirt Added To Cart"
}
```

Order & Payment APIs

Place COD Order

Method: POST

Endpoint: /api/order/place

Description: Places an order via Cash On Delivery. Deducts inventory atomically to prevent race conditions. Applies coupons if valid.

Authentication: Required

Authorization: User

Request Body:

```
{
  "items": [
    {
      "_id": "60d21b46...",
      "name": "Classic T-Shirt",
      "size": "M",
      "quantity": 2
    }
  ],
  "address": {
    "name": "Home",
    "phone": "9876543210",
    "addressLine1": "123 Street",
    "city": "Chennai",
    "state": "TN",
    "postalCode": "600001",
    "country": "India"
  },
  "couponCode": "SAVE20"
}
```

Success Response:

```
{
  "success": true,
  "message": "Order Placed"
}
```

Place Stripe Order

Method: POST

Endpoint: /api/order/stripe

Description: Initializes a Stripe Checkout session. Automatically factors in dynamic product pricing, delivery fees, and generates on-the-fly Stripe coupons if a discount code is applied.

Authentication: Required

Authorization: User

Success Response:

```
{
  "success": true,
  "session_url": "https://checkout.stripe.com/pay/cs_test_..."
}
```

Verify Stripe Payment

Method: POST

Endpoint: /api/order/verifyStripe

Description: Verifies the Stripe payment redirect. If successful, deducts inventory and marks order as paid.

Authentication: Required

Authorization: User

Request Body:

```
{
  "orderId": "60d21b46...",
  "success": true
}
```

Coupon APIs

Validate Coupon

Method: POST

Endpoint: /api/coupon/validate

Description: Checks if a provided discount code is valid, active, and meets minimum order criteria.

Authentication: Required

Authorization: User

Request Body:

```
{
  "code": "WELCOME50",
  "subtotal": 1500
}
```

Success Response:

```
{
  "success": true,
  "discountPercentage": 10,
  "maximumDiscountAmount": 500
}
```

Admin APIs

The Admin APIs require the `adminAuth` middleware (Token generated via `ADMIN_EMAIL` + `ADMIN_PASSWORD`).

Add Product

Method: POST

Endpoint: `/api/product/add`

Description: Creates a new product with multiple variants/inventory arrays and uploads up to 4 images via Multer/Cloudinary.

Headers: Content-Type: multipart/form-data, token: <Admin-Token>

Update Order Status

Method: POST

Endpoint: `/api/order/status`

Description: Admin updates order tracking status (e.g., "Order Placed", "Shipped", "Delivered").

Request Body:

```
{
  "orderId": "648a1b2c3d...",
  "status": "Shipped"
}
```

Dashboard Statistics

Method: GET

Endpoint: `/api/admin/dashboard`

Description: Aggregates order volume, revenue, customer count, and product count for the admin dashboard.

8. Error Handling

The API always returns a consistent JSON object structure upon error, maintaining an HTTP Status of 200 for graceful frontend interception.

Common Error Structure:

```
{
  "success": false,
  "message": "Error description here"
}
```

Common Errors Triggered:

- **Authentication:** "Not Authroized Login Again", "Please login first to place order!"
- **Validation:** "Invalid email format", "Password must be at least 8 characters long"
- **Inventory/Cart:** "Size is out of stock.", "Only X items are available for size Y."
- **Uploads:** "Image must be 5 MB or smaller.", "Unsupported image format."

9. Postman Testing Guide

1. Base URL Detection:

- Local: `http://localhost:4000` (or actual PORT defined in `.env`)
- Production: Deployed URL (e.g., Render/Railway)

2. Testing Sequence:

1. POST `/api/user/register` -> Copy token
2. Paste token into Headers (token: <your_token_here>) for subsequent User routes.
3. POST `/api/user/admin` -> Copy admin token.
4. Paste admin token into Headers (token: <admin_token>) to test `/api/product/add` (requires Form-Data).
5. Use GET `/api/product/list` to fetch products.
6. Use POST `/api/cart/add` to test inventory handling.

10. API Workflow

User Registration/Login ↓ Browse Products (GET `/api/product/list`) ↓ View Product (POST `/api/product/single`) ↓ Upload Design (POST `/api/customizer/upload`) ↓ Add Custom T-Shirt to Cart (POST `/api/cart/addStandaloneCustomized`) ↓ Checkout (POST `/api/order/stripe`) ↓ Verify Payment (POST `/api/order/verifyStripe`)

11. Implementation Status

Module	Implemented	Notes
User Registration & Login	Implemented	Uses bcrypt and JWT
Product Browsing	Implemented	Search/Filter logic handled on frontend via List API
Product Details	Implemented	Single Product API functional
T-shirt Customization	Implemented	Cloudinary upload & Canvas Object saving active
Cart Management	Implemented	Strictly validates inventory at cart addition
Checkout & Payments	Implemented	Stripe & COD fully functional with atomic deduction
Order History	Implemented	User Orders API functional
Customer Profile & Address	Implemented	Default Address logic implemented
Admin Authentication	Implemented	Independent Admin Login via Environment Variables
Product & Inventory Mgmt	Implemented	Advanced inventory arrays (size, colour, qty)
Design Library	Implemented	Admin can manage predefined Customizer designs
Coupons/Discounts	Implemented	Validation & dynamic cart calculation active
Homepage Content	Implemented	Dynamic Banners & Content configurable via Admin
Category APIs	Not Implemented	Stored natively on products, no standalone category API

Note: The Category APIs are marked as Not Implemented because categories and subcategories are directly handled as strings in the `productModel` rather than via independent Category REST endpoints. This aligns with the implemented logic.